PARSER

GTECH 3 - 2025/2026

PRESENTATION

PROGRAMME

SEMAINE THÉORIQUE	03-11-2025 -> 07-11-2025
SEMAINES PROJET	10-11-2025 -> 14-11-2025

THÉORIQUE

PARSER GENERALISTE

- Parser Texte simple / avancé
- Parser Binaire
- Parser .obj
- Introduction sur le sdk fbx

OPENGL

- Présentation/ Introduction d'opengl
- Présentation / Explication de minigl

PARSER TEXTUEL

PARSER SIMPLE

Imaginez un fichier de configuration qui ressemblerait à :

propriety01:value01
propriety02:value02
propriety03:value03

Quelle structure mémoire pouvons nous utiliser ?
Quelle logique ?

PARSER MOINS SIMPLE

Imaginez un fichier de configuration qui ressemblerait à :

propriety01:value01
propriety02:[valueArr01, valueArr02, valueArr03]
propriety03:value02
//propriety04:value03

Quelle structure mémoire pouvons nous utiliser ?
Quelle logique ?

LEXER

En informatique, l’analyse lexicale, lexing ou tokenization est la conversion d’une chaîne de caractères (un texte) en une liste de symboles (tokens en anglais).
Elle fait partie de la première phase de la chaîne de compilation. Ces symboles sont ensuite consommés lors de l'analyse syntaxique.
Un programme réalisant une analyse lexicale est appelé un analyseur lexical, tokenizer ou lexer.
Un analyseur lexical est généralement combiné à un analyseur syntaxique pour analyser la syntaxe d'un texte. (Merci wikipedia)

PARSER MOINS SIMPLE

Toujours ce fichier:

propriety01:value01
propriety02:[valueArr01, valueArr02, valueArr03]
propriety03:value02
//propriety04:value03

Quels seront les symboles qu'on va devoir utiliser ?

EXERCICE

ON LE CODE ?

- Reprendre l'exemple du slide précédent.
- Faire le Tokenizer
- Faire l'analyse des tokens pour tout récupérer en mémoire.

Vous avez fini avant la pause ?

- Proposer moi la liste des tokens qui seront utile pour parser un .json.

PARSER BINAIRE

PARSER UN .BMP

Information sur la structure d'un .bmp: https://en.wikipedia.org/wiki/BMP_file_format

On peut loader directement les informations dans des structures non alignée en mémoire pour stocker et lire les informations.

Si on veut lire le header "Bitmap file header", le code peut ressembler à ca:

(attention à l'endian de votre os / fichier)

```cpp
#include <iostream>
#include <fstream>

struct __attribute__((__packed__)) BMPFileHeader {
    unsigned char fileMarker[2];
    unsigned int size;
    uint16_t unused1;
    uint16_t unused2;
    unsigned int imageDataOffset;
};

int main() {
    std::ifstream fs;
    fs.open( s: "assets/chat.bmp", mode: std::ios::binary | std::ios::in);

    BMPFileHeader bh;

    fs.read(reinterpret_cast<char *>(&bh), n: sizeof(BMPFileHeader));
    std::cout << bh.size << std::endl;

    return 0;
}
```

EXERCICE

ALLEZ ON LE CODE

- Reprendre la base du code précédent.
- Ajouter la lecture du header info.
- Lire et stocker la liste des pixels de l'image
- Afficher les grâce à une texture/sprite sfml.